Final Report

Team #7
Team Members: Adrien Rozario, Vonn Sayasa, Grace Yu, Tamiya Phillips
Project Title: VitaMetrics: Personal Health Dashboard

1. Executive Summary

VitaMetrics is a cloud-based personal health tracking platform that allows users to monitor and visualize key wellness metrics such as sleep, exercise, nutrition, mood, and stress all in one centralized dashboard. The application was developed to provide a more accessible and holistic alternative to fragmented health tracking tools and expensive wearable-device ecosystems. The final system includes user authentication, cloud-hosted APIs, interactive visualizations, goal tracking, file uploads, Azure cloud deployment, and persistent storage using Azure Cosmos DB and Azure Blob Storage.

2. Problem Statement & Solution Description

Adults with busy schedules, students balancing academic responsibilities, and individuals managing chronic health conditions often lack a centralized and accessible way to monitor their overall wellness outside of occasional doctor's visits. Many modern health tracking platforms focus only on isolated aspects of wellness, such as calorie tracking, exercise logging, or wearable device integration. This creates a fragmented user experience where individuals struggle to understand how different lifestyle factors interact with one another. Additionally, many advanced health monitoring ecosystems rely heavily on smartwatches or fitness devices, creating financial and accessibility barriers for users who want a simpler solution.

VitaMetrics was designed to solve this problem by providing users with a centralized, cloud-based health dashboard that combines multiple dimensions of wellness into a single platform. Rather than requiring wearable devices, the application allows users to manually log health information and visualize long-term patterns through interactive charts and summaries.

The final prototype includes several major features. Users can create accounts and securely log into the application using Supabase Authentication. Once authenticated, users can navigate through different sections of the platform using a sidebar navigation system.

The Dashboard page serves as the central analytics hub of the application. Users can dynamically filter data using predefined ranges such as the last 7 days, the last 30 days, or custom date ranges. These filters update all visualizations simultaneously, allowing users to analyze their habits over different periods of time.

The dashboard includes:

- A line chart for sleep trends over time
- A bar chart for exercise activity
- A scatter plot comparing wellness metrics such as mood and stress against sleep or exercise
- A donut chart visualizing macronutrient distribution
- Goal progress indicators and summary statistics

Users can also log detailed health metrics through the Log Metrics page. Inputs include sleep hours, sleep quality, exercise type and duration, mood level, stress level, calorie intake, protein, carbohydrates, and fat intake.

The Goals page allows users to create personalized wellness goals for sleep, exercise, and protein intake. Progress bars and completion charts dynamically compare current performance against user-defined targets.

The Files page enables users to upload meal images and health-related documents. Uploaded files are stored in Azure Blob Storage while metadata references are stored in Cosmos DB. Users can retrieve and view uploaded files directly within the application.

Some originally proposed stretch features were not implemented due to time constraints and deployment complexity. These include automated wellness insights, notifications, reminder systems, advanced statistical analysis, and deeper health prediction features. The team prioritized delivering a stable and fully deployed cloud application with functional end-to-end workflows rather than overextending the project scope.

3. Architecture As-Built

The final VitaMetrics architecture evolved significantly from the original proposal. Initially, the system was planned around a more relational SQL-style structure with separate tables for sleep, exercise, nutrition, wellness, goals, and files. However, during implementation, the team transitioned to a NoSQL document-based architecture using Azure Cosmos DB.

The deployed architecture consists of five major components:

1. React Frontend
2. Express Backend API
3. Azure Cosmos DB
4. Azure Blob Storage
5. Supabase Authentication

The frontend was built using React Router, TypeScript, Tailwind CSS, and Recharts. The application provides a modern single-page experience while using server-side rendering for

deployment compatibility and routing support. The frontend handles all user interaction, dashboard rendering, form submission, filtering, and navigation.

The backend was implemented using Node.js and Express. The API layer validates requests, processes metric submissions, queries Cosmos DB, handles file uploads, and generates responses consumed by the frontend. Rather than using many isolated endpoints, the team ultimately adopted a unified /api/metrics endpoint that stores all health metrics for a user and date within a single document.

Azure Cosmos DB serves as the primary database for structured health data. The system uses multiple collections:

- Users
- Metrics
- Goals
- Files

The Metrics collection became the core of the application. Instead of using separate relational tables, each document contains embedded sub-objects for sleep, exercise, nutrition, and wellness data. This flexible structure allowed users to log only the metrics they wanted without requiring rigid schemas.

Azure Blob Storage is used for object storage. Meal images and uploaded health reports are stored as blobs, while Cosmos DB stores metadata references such as file type, upload date, and blob paths. The backend generates secure SAS URLs that allow the frontend to retrieve and display files.

Supabase Authentication was used to manage account creation, login, session management, and user authentication. Originally, the project planned to build authentication manually using Firebase and password hashing. However, the complexity of integrating and maintaining Firebase within the Azure deployment pipeline led the team to adopt Supabase instead. This significantly simplified authentication workflows while improving reliability and development speed.

The entire system was deployed using Azure App Service and connected to GitHub Actions CI/CD workflows. Every push to the main branch automatically rebuilt and redeployed the application to Azure. The final deployed application is hosted at:

https://v1tametrics.azurewebsites.net/

4. Technical Decisions

One of the most significant technical decisions was switching from a SQL-style relational design to a NoSQL document-based architecture using Azure Cosmos DB. Initially, the team

planned to use separate relational-style tables for different metric categories. However, during implementation, the team realized that user health logs varied significantly in structure and that enforcing rigid schemas created unnecessary complexity.

By consolidating all daily metrics into a single document per user per day, the backend became simpler and more scalable. The NoSQL approach also allowed optional fields and flexible nested structures, which fit naturally with wellness tracking workflows.

Another important decision involved backend hosting. The team initially explored Azure Functions but later transitioned to Azure App Service running a Node.js Express server. Azure Functions introduced deployment complications and multipart form parsing issues during file upload integration. App Service provided a more stable environment for handling SSR routing, API endpoints, Blob Storage integration, and frontend hosting within a unified deployment workflow.

The team also chose Supabase Authentication instead of building a custom authentication system. While implementing authentication manually would have offered more control, Supabase dramatically reduced development time and simplified session handling, user persistence, and frontend integration.

React Router with SSR support was selected as the frontend framework because it allowed the frontend and backend to coexist cleanly within the same deployment environment. Tailwind CSS was used for styling because it enabled rapid UI iteration while maintaining a consistent design system.

The Recharts visualization library was chosen because it integrated naturally with React and allowed the team to quickly implement dynamic visualizations such as line charts, bar charts, scatter plots, and donut charts without requiring low-level chart configuration.

5. Challenges & Resolutions

One of the most significant challenges was the transition from a SQL-style relational structure to a NoSQL document-based architecture using Azure Cosmos DB. The original proposal separated health metrics into independent relational-style tables. However, implementing this structure within Cosmos DB created unnecessary complexity and increased the number of required API calls. To simplify the architecture, the team redesigned the data model so that each user would have a single document per date containing embedded health metrics such as sleep, exercise, nutrition, and wellness data. This dramatically simplified querying, filtering, and dashboard rendering while also providing greater flexibility for optional fields and future feature expansion.

Another major challenge involved authentication. The project originally planned to use Firebase Authentication alongside custom backend logic. However, integrating Firebase cleanly with the evolving Azure deployment architecture proved difficult. Session handling, deployment

environment variables, and frontend/backend synchronization introduced repeated authentication failures during development and testing. The team ultimately resolved this issue by migrating to Supabase Authentication, which provided a more streamlined integration process and significantly improved authentication reliability and session management.

Deployment and hosting presented some of the most difficult technical challenges throughout the project. The team encountered repeated issues involving multiple local servers, conflicting frontend/backend routing systems, React Router SSR build outputs, and Azure deployment failures. During deployment, the application would build successfully through GitHub Actions but repeatedly fail to launch in Azure App Service. Both Linux- and Windows-based Azure environments were tested, but the issue persisted.

After extensive debugging through Azure log streams, the team discovered that the "node_modules" directory was unexpectedly being deleted and recreated during deployment. This caused the application to attempt startup without required dependencies, resulting in continuous reboot loops and failed launches. The issue was ultimately resolved through a combination of modifying the GitHub Actions workflow to ensure required dependencies were properly included during deployment and configuring a custom startup command provided during troubleshooting sessions with the course staff.

Additional deployment issues occurred because frontend assets failed to load correctly due to misconfigured Express routing and React Router SSR output paths. The team also encountered confusing conflicts caused by stale local server instances during testing. These issues were resolved by restructuring the backend server configuration, standardizing API paths across the frontend and backend, and refining the automated GitHub Actions deployment workflow.

File uploads introduced another layer of technical complexity. The application initially experienced issues where uploaded images and documents failed to render correctly after deployment. Integrating Azure Blob Storage required debugging environment variables, SAS URL generation, multipart form handling, and synchronization between Blob Storage metadata and Cosmos DB references. After several iterations of debugging and restructuring the upload workflow, the final implementation successfully stored uploaded files in Azure Blob Storage while maintaining retrievable metadata references inside the Files collection.

A final challenge involved project scope management. The original proposal envisioned advanced analytics, automated insights, notifications, and more sophisticated health recommendation features. As the semester progressed, the team recognized that ensuring a stable and fully deployed cloud application was more important than implementing every proposed stretch feature. The team intentionally narrowed the project scope to prioritize reliability, deployment stability, authentication, cloud integration, and overall user experience.

6. Division of Work

Adrien Rozario - Cloud Architect & Backend Lead:

Adrien designed the overall cloud architecture and backend API structure. Responsibilities included implementing Express API routes, integrating Cosmos DB, configuring Azure Blob Storage workflows, debugging SSR deployment issues, restructuring the database model from SQL-style tables to NoSQL documents, integrating frontend-backend communication, implementing dashboard filtering logic, and assisting with deployment troubleshooting.

Vonn Sayasa - DevOps & Automation Lead:

Vonn provisioned Azure cloud resources, configured Azure App Service deployment, implemented GitHub Actions CI/CD workflows, managed environment variables and deployment secrets, and led debugging efforts involving Azure hosting, SSR deployment configuration, routing conflicts, and deployment automation.

Grace Yu - Research & Database Engineering Lead:

Grace designed the database architecture, helped restructure the application around Cosmos DB collections, contributed to data validation, refined document structures for metrics and goals, and assisted with dashboard feature integration and data organization.

Tamiya Phillips - Frontend & UX Lead:

Tamiya designed and implemented the frontend interface, including navigation systems, dashboard layouts, user workflows, chart integration, and responsive styling. She also worked on the Goals page, visualization design, and overall UI consistency across the platform.

7. Reflection

Overall, the project successfully achieved its primary objective of building and deploying a fully functional cloud-based health tracking platform. The team's biggest accomplishment was integrating multiple cloud technologies into a cohesive system that included authentication, database storage, object storage, deployment automation, and interactive frontend visualizations.

One thing that went particularly well was the team's ability to adapt when the original architecture became impractical. Transitioning from relational-style structures to a NoSQL document model improved both flexibility and scalability. The team also gained valuable experience working with real-world cloud deployment workflows rather than purely local development environments.

If the project were restarted, the team would likely simplify the architecture earlier in development and establish deployment infrastructure sooner. Many debugging issues arose because frontend and backend hosting strategies evolved while development was already underway. Earlier deployment testing would have prevented several integration problems later in the semester.

The project also changed the team's understanding of cloud computing. Initially, cloud infrastructure seemed primarily focused on hosting applications. However, throughout development, the team learned that cloud computing also involves deployment automation, scalability planning, environment variable management, authentication integration, storage architecture, debugging distributed systems, and coordinating multiple services together.

One of the most valuable lessons learned was that cloud development is often less about writing isolated code and more about understanding how interconnected systems behave together. While many technical challenges arose throughout the semester, successfully resolving those issues provided a much deeper understanding of modern full-stack cloud application development.